Lec 73
Samarth Sir

4 Jan 2025

useRef() → ye bhi ek hook hai.

The useRef hook is a built-in hook in React that provides a way to persist mutable values across renders without causing re-renders. It can also be used to directly interact with DOM elements.

use case 1 of useRef() :-

→ DOM ke elements ka reference mil jata hai iska use karke. you will be able to access DOM elements using useRef() and manipulate that selected DOM element.

use case 2 of useRef() :-

ye state ko maintain karne mei kaam aata hai between two renders.

maan lo aapne application chaalu kari then first time render hoga. Agar koi state variable change hua, then component rerender hoga and agr chahte ho ki koi state variable preserved/maintained rahe between first & second render then you will need to use useRef().

preserve/maintain ka matlab hai ki state variable ki value same rahe.

```
import React from 'react'

function Hookss() {

    let salary = 50000;

  return (
    <div>
        <h1>salary is: {salary}</h1>
    </div>
  )
}

export default Hookss
```

↳ mai chahta hu ki iss code mei 50000 ki jagah web page par 33k aajaye but after 3 seconds.

SetTimeout() use karna padega; matlab useEffect() use karna padega.

```
import React, { useEffect } from 'react'

function Hookss() {

    let salary = 50000;

    useEffect(function(){
        setTimeout(()=>{
            document.getElementById('sal').innerText = '33k';
        }, 3000)
    },
    [])

  return (
    <div>
        <h1 id = 'sal'>salary is: {salary}</h1>
    </div>
  )
}

export default Hookss
```

Ab 3sec ke baad 50000 ki jagah 33k aayega webpage par. Humne salary ko state variable nahi banaya. We kept it as a normal variable. Humne element ko select karke innerText change kar diya. Instead

Up making salary a state variable.

```
document.getElementById.innerText =
```

↳ itni lambi line likhne se bachne ke liye hi to hamne React seekhna shuru kiya tha.

Better alternative :- use karo useRef() ko

Sabse pehle useRef() hook ko initiate karo.

↳ Step1 : let spanEl = useRef();

Step2 : ↓

```
<div>
    Salary is : <span ref={spanEl}> {salary} </span>
</div>
```

 tag ke andar hamara {salary} hai isliye ref = {spanEl} tag ke andar likho. useRef() points to reference of DOM element.

Now tag will point to "spanEl" by writing the line.

Ab hamare paas reference aa gya. Ab hame reference ki value ko change karna hai.

Step3 :-

```
useEffect(function(){
    setTimeout(()=>{
        spanEl.current.innerHTML = "33000";
    }, 3000)
},
[])
```

↳ dom manipulation

spanEl.current.innerHTML = "33000";

```
import React, { useEffect, useRef } from "react";

function Hookss() {

    let salary = 50000;
    let spanEl = useRef();

    useEffect(function(){
        setTimeout(()=>{
            spanEl.current.innerHTML = "33000";
        }, 3000)
    },
    [])

  return (
    <div>
        Salary is : <span ref={spanEl}> {salary} </span>
    </div>
  )
}

export default Hookss
```

→ final code

useRef() allows direct DOM manipulation w/o triggering re-renders.

3. **Why `useRef`?**
   - `useRef` allows direct DOM manipulation without triggering re-renders.
   - Here, it changes the content of the `<span>` without needing state or re-rendering.

Why Not `useState`?

- If you used `useState`, changing the value would trigger a re-render.
- Here, `useRef` is ideal because we only need to modify the DOM directly without affecting the React rendering flow.

3. `ref={spanEl}`

- **Purpose**: This connects the `<span>` element to the `useRef` reference (`spanEl`).
- **How It Works**:
  - `useRef()` creates an object: `{ current: null }`.
  - When the `<span>` element is rendered, React sets `spanEl.current` to the DOM node of this `<span>`.
  - You can now interact directly with this DOM element using `spanEl.current`.

Samarth sir ne bola ki development mei hum useRef() ka use case 1 hi use karte hai naaki use case 2.

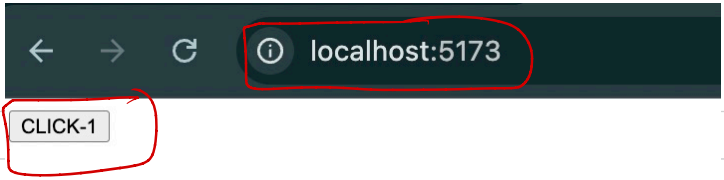
Q guess the output :-

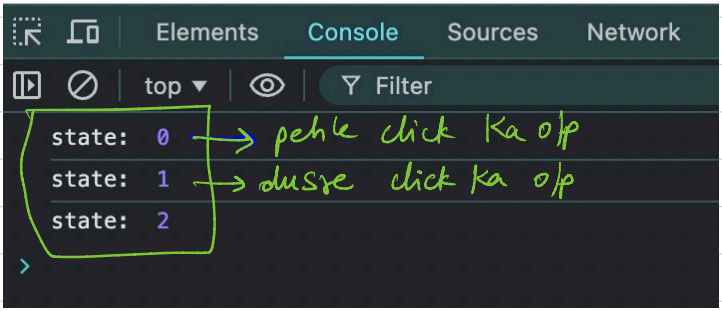
```
import React, { useRef, useState } from "react";

function CaseTwo() {
    const [count, setCount] = useState(0); → l0

    function handleIncrement() {
      setCount(count + 1); // 0 + 1 = 1 //next render-2
      console.log("state: ", count); // 0 //render-1
    }

    return (
      <div>

        <button onClick={handleIncrement}>CLICK-1</button>

      </div>
    );
  }

  export default CaseTwo;
```

l1 ←
l2 ←

jab mai button par click karenga tab o/p 1 hona chahiye but 0 aayega kyuki l1 time lega and l2 turant chal jayega.

Note:- l0 par hamne const use kiya hai. Agar let use karte tab bhi code same tareeke se chalta.



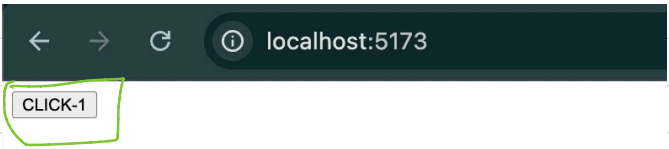


Also useState() ke case mei hamne dekha ki state variable change ho raha hai; pehle 0 tha, fir 1 ho gya. Rerendering hone par state variable ki value preserve nahi rahi.

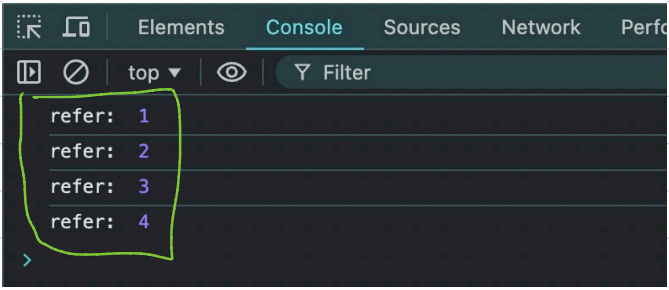
Now let's use useRef() :-

```jsx
import React, { useRef, useState } from "react";

function CaseTwo() {

    let countRef = useRef(0);
    function handleIncrement2() {
      countRef.current += 1;
      console.log("refer: ", countRef.current);
    }

    return (
      <div>

      <button onClick={handleIncrement2}>CLICK-1</button>

      </div>
    );
  }

  export default CaseTwo;
```

l0 ← l1 → l2 → l3 →



yaha har click par value change hora hai.



How the value / state of countRef is being preserved.

Why Is the Value "Preserved" Even When It Changes?

1. `useRef` **Creates a Persistent Object:**
   - When `useRef(0)` is called, React creates an object:

     ```javascript
     { current: 0 }
     ```

     Copy code
   - This object is stored internally by React and survives the component's re-renders.
2. **Modifying the Value:**
   - When you update `countRef.current`, you're not replacing the reference object; you're only modifying its `current` property.
   - React doesn't reset or recreate the `countRef` object during re-renders. It remains the same object throughout the component's lifecycle.
3. **Preservation Across Re-Renders:**
   - Since the same `useRef` object persists across re-renders, any changes made to `countRef.current` are retained and accessible in subsequent renders.
   - **The key here is that React doesn't overwrite or discard the `useRef` object on re-renders.**

use case 2 ka video you can watch again in free time for better understanding. Note that ki ye development mein itna use nahi hota.

Routing :- (very very important)

React ke paas khud ka router nahi hai.
It uses third party library for the same.

Name :- React Router DOM
[reactrouter.com]

we will need to install it:

npm i react-router-dom

(56:50) Hum jaante hai ki React Single Page Application hai. Ab routing use karne ke baad bhi, it should stay SPA → single page application.

Client side Bundling ko client side Rendering kehte hai.

React mei refreshing nahi hoti ; isliye sabhi pages client side per bante hai.

define client side bundling :- Maanlo aapki application mei 3 pages hai. Aap ke 1 server call per sara JS bundle hoke aa jata hai. Isliye jab aap 3 different pages / routes hit karoge, tab reloading / refreshing nahi hogi.

Let's assume your application has three routes:

- `/` (Home page)
- `/about` (About page)
- `/contact` (Contact page)

Initial Load:

- When the user visits `/`, the browser fetches:
  - The main HTML file (e.g., `index.html`).
  - The initial JavaScript bundle (e.g., `main.js`), which contains only the code necessary for the Home page.

Navigating to `/about`:

- When the user clicks on a link to `/about`, React Router detects the route change.
- If code splitting is enabled:
  - The JavaScript chunk for the About page (e.g., `about.chunk.js`) is fetched from the server.
  - The About page is rendered **without reloading the entire application.**

Navigating to `/contact`:

- Similarly, when the user clicks on a link to `/contact`, the JavaScript chunk for the Contact page (e.g., `contact.chunk.js`) is fetched and rendered dynamically.

Each of these JavaScript chunks is **bifurcated** (separated) from the main bundle to optimize performance.

---

1. What Does "Bifurcated from JS Data Bundler" Mean?

When you navigate to the other two pages in your React app, their specific JavaScript code (e.g., the code for the `About` or `Contact` page) is fetched separately from the server **on demand** rather than being loaded all at once in the initial JavaScript bundle.

This happens because React and bundling tools (like Webpack or Vite) allow dividing your application into smaller chunks or bundles that can be loaded only when needed. This process is known as **code splitting**.

Client side Bundling & client side rendering :-

Relation Between the Two

1. **Bundling** prepares the JavaScript code, assets, and dependencies into optimized packages for delivery to the client.
2. **Rendering** uses these bundled files to dynamically generate the user interface in the browser.

Client-side rendering (CSR) is the process of rendering web pages on the client using JavaScript. In this approach, the server sends the initial HTML file, but the client then uses JavaScript to dynamically update the page as needed. This allows for more interactive and responsive web pages, as the client can update specific parts of the page without needing to reload the entire page.

One example of a popular CSR framework is React. With React, you can write JavaScript code that updates the DOM as needed, providing a more interactive and dynamic web application.

Working of CSR: When a user requests a page, the server sends the initial HTML file, along with any required JavaScript files. The client then uses JavaScript to update the page as needed, without needing to reload the entire page.

Uses: CSR is commonly used for web applications that require a high degree of interactivity, such as social media platforms or e-commerce websites.

"React-continued" folder mei "npm i react-router-dom" likho

App.jsx mei return mei ye likho :-

<BrowserRouter>
</BrowserRouter>

→ it is a parent wrapper of all routes. It is compulsory to use it.

and header mei likho :-

import {BrowserRouter} from "react-router-dom"

<Routes> </Routes> ⇒ sabhi routes isme wrap hoge.

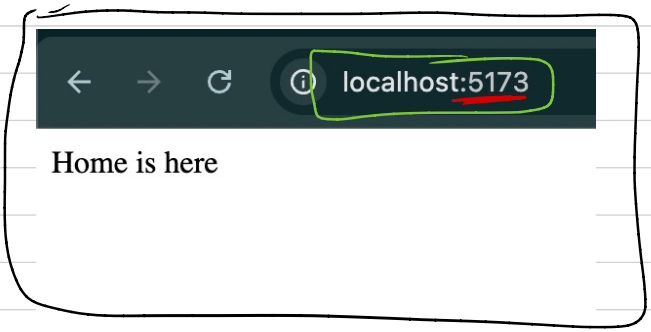
```
import React from 'react'
import Hookss from './Components/Hookss'
import CaseTwo from './Components/CaseTwo'
import { BrowserRouter, Routes} from "react-router-dom";

function App() {
  return (
    <div>

      <BrowserRouter>
        <Routes>

        </Routes>
      </BrowserRouter>

    </div>
  )
}

export default App
```

Ab <Routes> ke ander hum <Route> likhenge.
We will write routes inside <Route> </Route>.

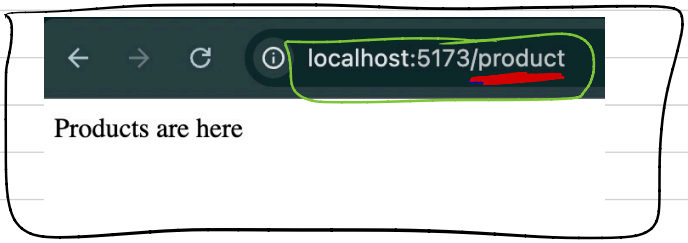
```
import React from 'react'
import Hookss from './Components/Hookss'
import CaseTwo from './Components/CaseTwo'
import { BrowserRouter, Routes, Route} from "react-router-dom";

function App() {
  return (
    <div>
      {/* <Hookss/> */}
      {/* <CaseTwo /> */}
      <BrowserRouter>
        <Routes>

          <Route path = '/' element= {<Home/>} />
          <Route path = '/product' element= {<Product/>}/>

        </Routes>
      </BrowserRouter>

    </div>
  )
}

export default App
```

Routing set ho gyi.

Ab Home.jsx and Products.jsx banao.



→ different route par different o/p




```jsx
import React from "react";

function Home() {
  return <div>Home is here</div>;
}

export default Home;
```

→ Home.jsx

```jsx
import React from "react";

function Products() {
  return <div>Products are here</div>;
}

export default Products;
```

→ Products.jsx

ab hum frontend wali bhi routing wala kaam kar pa rahe hai.

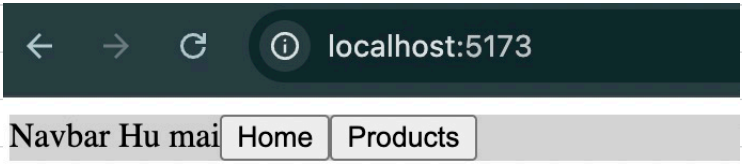
```
import React from 'react'
import Hookss from './Components/Hookss'
import CaseTwo from './Components/CaseTwo'
import Products from "./Components/Products"
import Home from "./Components/Home"
import { BrowserRouter, Routes, Route} from "react-router-dom"


function App() {

  function handleClick1() {
    window.location.href = '/';
  }

  function handleClick2() {
    window.location.href = '/product';
  }

  return (
    <div>
      {/* <Hookss/> */}
      {/* <CaseTwo /> */}

      <div style= {{ backgroundColor: "lightgray", color: "black" }}>
      Navbar Hu mai
      <button onClick = {handleClick1} >Home </button>
      <button onClick = {handleClick2} >Products </button>
      </div>


      <BrowserRouter>
        <Routes>

            <Route path = '/' element= {<Home/>} />
            <Route path = '/product' element= {<Products/>}/>

        </Routes>
      </BrowserRouter>

    </div>
  )
}

export default App
```

Note:- <BrowserRouter> se pehle hamne element use kiya hai. Check l2.

Hamne do buttons add kar diye jinke click hone par routes change hoge.

window.location.href ⇒ helps with route changing.

↳ ye window hata dege tab bhi chalega.



Mai jaise hi "Products" wale button par click karunga, waise hi "/products" ka route aa jayega.

But React Single Page application banata hai; hamare case mei button click hone par refresh ho raha hai.

location.href ⇒ is doing refreshing here

We will use useNavigate() ⇒ it is a hook.

↳ provided by react-router-dom.

```
function App() {

  let navigate = useNavigate();
  function handleClick1() {

     navigate('/');
  }

  function handleClick2() {

    navigate('/product');
  }

  return (
    <div>
      {/* <Hookss/> */}
      {/* <CaseTwo /> */}

      <div style= {{ backgroundColor: "lightgray", color: "black" }}>
      Navbar Hu mai
      <button onClick = {handleClick1} >Home </button>
      <button onClick = {handleClick2} >Products </button>
      </div>


      <BrowserRouter>
        <Routes>

            <Route path = '/' element= {<Home/>} />
            <Route path = '/product' element= {<Products/>}/>

        </Routes>
      </BrowserRouter>

    </div>
  )
}
```

l0

l1

l2

↳ yecode error deya due to l0, l1, l2.

localhost:5173

Elements Console Sources Network Performance Memory Components Applicat

top Filter

Uncaught Error: useNavigate() may be used only in the context of a <Router> component.
at App (App.jsx:61:18)
Show ignore-listed frames

Uncaught Error: useNavigate() may be used only in the context of a <Router> component.
at App (App.jsx:61:18)
Show ignore-listed frames

The above error occurred in the <App> component:

at App (http://localhost:5173/src/App.jsx?t=1736061964789:26:18)

Consider adding an error boundary to your tree to customize error handling behavior.
Visit https://reactjs.org/link/error-boundaries to learn more about error boundaries.

Uncaught Error: useNavigate() may be used only in the context of a <Router> component.
at App (App.jsx:61:18)
Show ignore-listed frames

Error kch saha hai's—

useNavigate() hook ko aap kahi bhi use nahi kar sakte; it has to be used inside <Router> component. In our case, <BrowserRouter> is that component.

react-router-dom ke sabhi elements/hooks (jaise useNavigate) sabka maalik ek hai → <BrowserRouter>

So, react-router-dom ki chije should be used inside <BrowserRouter> always.

```
function App() {


  return (
    <div>

      <BrowserRouter>

        <Nav />

        <Routes>

            <Route path = '/' element= {<Home/>} />
            <Route path = '/product' element= {<Products/>}/>

        </Routes>
      </BrowserRouter>

    </div>
  )
}


function Nav() {

  let navigate = useNavigate();
  function handleClick1() {

     navigate('/');

  }

  function handleClick2() {

    navigate('/product');

  }

  return (
    <div>

      <div style= {{ backgroundColor: "lightgray", color: "black" }}>
      Navbar Hu mai
      <button onClick = {handleClick1} >Home </button>
      <button onClick = {handleClick2} >Products </button>
      </div>

    </div>
  );
}

export default App
```

App.jsx

l0

l1

l0 → par hamne Nav call kardiya.
<Nav/> is written inside <BrowserRouter>

(1:51:20)